

1. Maximum Subarray

Given an integer array `nums`, find the [subarray](#) with the largest sum, and return *its sum*.

Example 1:

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`

Output: 6

Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.

Example 2:

Input: `nums = [1]`

Output: 1

Explanation: The subarray `[1]` has the largest sum 1.

Example 3:

Input: `nums = [5,4,-1,7,8]`

Output: 23

Explanation: The subarray `[5,4,-1,7,8]` has the largest sum 23.

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

Follow up: If you have figured out the $O(n)$ solution, try coding another solution using the **divide and conquer** approach, which is more subtle.

PROGRAM

```
class Solution {  
  
    public int maxSubArray(int[] nums) {  
  
        int currentSum = nums[0];  
  
        int maxSum = nums[0];  
  
        for (int i = 1; i < nums.length; i++) {  
  
            currentSum = Math.max(nums[i], currentSum + nums[i]);  
  
            maxSum = Math.max(maxSum, currentSum);  
  
        }  
  
        return maxSum;  
  
    }  
}
```

}

OUTPUT

github - Search

vtu30221-cmyk/PST-programs

Remove Duplicates In Place

Maximum Subarray - LeetCode

Unit 1 Weekly task.pdf

https://leetcode.com/problems/maximum-subarray/submissions/2145298780/

Problem List

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted 210 / 210 testcases passed

VTU30221 submitted at Sep 18, 2026 08:56

School's in. Leet's on.

Get LeetCode Annual Premium for \$99!

Runtime

1 ms | Beats 99.93%

Memory

77.14 MB | Beats 66.36%

Code | Java

class Solution {

Code

Auto

int maxSum = nums[0];

for (int i = 1; i < nums.length; i++) {

currentSum = Math.max(nums[i], currentSum + nums[i]);

maxSum = Math.max(maxSum, currentSum);

}

return maxSum;

Testcase

Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

nums =

[-2,1,-3,4,-1,2,1,-5,4]

Output

6

Expected

Search

ENG IN

82%

08:56 18-09-2026